

Details Where They Matter: Understanding Projection Spaces Using In-Place Summary Visualizations

Christian A. Steinparz , Yannick Metz , Andreas Hinterreiter , Manuela Waldner ,
Mennatallah El-Assady , and Marc Streit 

Abstract—Understanding the high-dimensional data behind 2D projections requires access to details, but prior approaches either demand tedious manual selection of items and groups or show all details at once, which scales poorly and overwhelms users. We present a method for adaptively showing details in place within scatterplots of projected high-dimensional data. Our main contribution is a workflow in which a degree of interest (DoI), derived from user interactions, automatically controls insets of summary and difference visualizations placed directly in the projection, yielding an annotated, map-like representation for trajectory data as well as unconnected items. Our approach derives the DoI from queries, selections, and semantic zooming, propagates it along spatial and topological similarity, and selects and places insets at the resulting level of aggregation and detail, all at interactive rates in a web-based prototype and pip-installable Jupyter widget. We evaluate our method against representative prior work in which projection views were labeled through manual or interaction-driven annotation. The results show that our solution replaces the repeated select-and-inspect loops necessary in other solutions entirely. Through automatic, progressive aggregation and annotations, our solution reveals details where they matter, on the fly and in the right context.

This paper and all supplemental materials are available at <https://jku-vds-lab.at/tvcg26-steinparz/material/>.

Index Terms—Degree of interest, semantic zoom, annotations, focus and context, trajectory data, dimensionality reduction, scatterplots

I. INTRODUCTION

LARGE-SCALE data analysis and machine learning increasingly involve datasets described by high-dimensional feature spaces, making direct visualization impractical. Dimensionality reduction (DR) techniques such as t-SNE [1] and UMAP [2] have therefore become popular for transforming such data into 2D projection spaces that support visual exploration. These projections reveal structures such as clusters and outliers by approximately preserving neighborhood relationships from the original feature space.

Christian A. Steinparz, Andreas Hinterreiter, and Marc Streit are with Johannes Kepler University Linz, 4040 Linz, Austria (e-mail: christian.steinparz@jku.at, andreas.hinterreiter@jku.at, marc.streit@jku.at).

Yannick Metz and Mennatallah El-Assady are with ETH Zürich, 8092 Zürich, Switzerland (e-mail: yannick.metz@inf.ethz.ch, melassady@ai.ethz.ch).

Manuela Waldner is with TU Wien, 1040 Wien, Austria (e-mail: manuela.waldner@tuwien.ac.at).

Manuscript submitted September 16, 2026.

However, understanding projection spaces remains difficult. Low-dimensional projections inevitably distort high-dimensional relationships, and scatterplots alone do not explain the high-dimensional characteristics underlying the visible patterns. Consequently, understanding what is important in a projection space and how much detail to show where remains a central challenge.

This sensemaking challenge becomes even more pronounced when topological relationships are introduced, as in trajectory data, where actions connect states into trajectories. This setting is broadly relevant because it applies whenever we want to make sense of sequential processes, including all problems that can be formulated as reinforcement learning environments.

Across research papers that contain (static) projection figures, authors often address this need for interpretation through *layout enrichment* [3], i.e., manually adding annotations such as text labels and representative examples. In interactive visualization systems, projection views are similarly complemented with linked views, hover details, selections, and other mechanisms for inspecting the underlying data. These existing approaches often rely either on tedious interaction workflows in which users repeatedly select and inspect subsets of points, or on dense visualizations that expose all information simultaneously, quickly leading to clutter and scalability problems. When labels and levels of aggregation are predefined, analysts can explore only the distinctions that are encoded, and targets outside this scope force them back into selection and inspection. These approaches therefore make limited use of the data structure and user intent to decide automatically what should be aggregated, annotated, or shown in detail.

To close this gap, we propose a workflow for exploring projection spaces in a way that resembles map-based exploration in systems such as Google Maps: users search, indicate what they consider relevant, and zoom between overview and detail while the visualization adapts what is shown.

Concretely, our workflow starts from a 2D projection as the spatial substrate for exploration. Users select items or query features, thereby assigning an initial degree of interest (DoI) [4] to the corresponding states. The system then automatically propagates this data DoI based on spatial proximity and trajectory topology, controlling the visual prominence of nodes and edges. Clustering then organizes annotation-

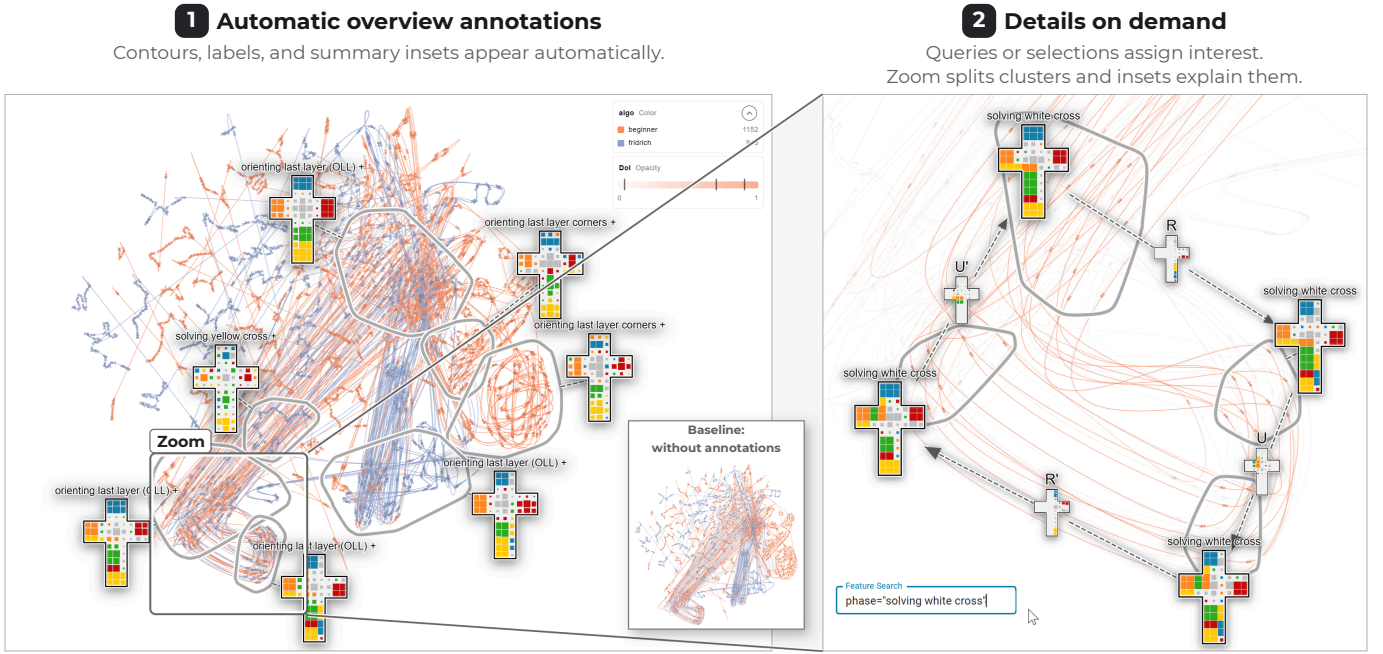


Fig. 1. Our method adaptively shows details in place within projections of high-dimensional data, here trajectories of Rubik’s cube solutions. (1) *Without any user input, the projection is annotated like a map* : clusters receive contours, text labels, and summary visualizations. For comparison, we show the baseline without annotations. (2) *Details appear on demand* : a degree of interest (DoI) derived from user interactions such as queries, selections, and semantic zooming adapts the level of detail in place. Zooming into the marked region splits clusters, refines insets, and adds difference insets that explain the moves between states.

eligible states into a multiscale hierarchy. A second, layout-based degree of interest ranks candidate clusters in the current 2D view by combining their propagated relevance with their current visual prominence, and a visual entity budget selects the highest-ranked candidates. For these clusters, the system generates labels and detail insets from the underlying data, bringing high-dimensional characteristics back into the scatterplot and revealing them directly within the projection. A final placement stage then determines and optimizes the positions of these annotations within the projection. As users select, pan, and zoom, these stages continuously update, creating a map-like representation of the projection space. Figure 1 shows an example result of our method applied to Rubik’s cube trajectories.

Our workflow is adaptive, updating annotation density, aggregation, and detail level to match the data, the current view, and the interests users specify. It is also adaptable, letting users steer this behavior through selections, queries, navigation, and a broad set of parameter controls. Together, these properties allow relevant details to appear where they matter, with little or no manual effort. Exploration proceeds by progressive disclosure, with annotations serving both as navigational cues and as the details themselves, refining as users zoom toward what interests them. Since the underlying hierarchy is generated during analysis rather than fixed in advance, this refinement can continue to arbitrary depth. And because details surface without being explicitly requested, exploration also invites serendipitous discovery.

In summary, this paper makes the following contributions:

- An adaptive and adaptable workflow that enriches projection scatterplots and trajectory charts with in-place

annotations (Section IV).

- A parametrized degree-of-interest framework that controls which parts to annotate based on user interaction, projection coordinates, topology information, and viewport constraints (Sections IV-A to IV-F).
- Open source implementations of the workflow as a web app and a [pip-installable Jupyter widget](#) , based on efficient DoI propagation and rendering algorithms (Section V).
- An evaluation showing that our approach can replace previously tedious interaction loops and manual annotations efficiently, and a runtime analysis of the workflow (Section VII).

This article extends an earlier workshop version of this work [5].

II. DATA MODEL

Our method operates on projected high-dimensional data that may additionally contain topological trajectory structure, represented in a tabular data format as shown in Figure 2. We treat the 2D projection as a fixed spatial substrate, where proximity serves as a proxy for similarity in the original feature space, agnostic of the dimensionality-reduction method used to obtain it. This setting relates to established projection-analysis tasks over projected points, including *discovering clusters*, *naming clusters*, and *discovering outliers*, alongside related neighborhood- and class-oriented tasks [3]. When trajectory structure is available, similar analytical questions may also concern transitions and paths, for example *discovering transition bundles*, *comparing trajectories*, or *identifying repeated trajectory patterns*. The remainder of this section

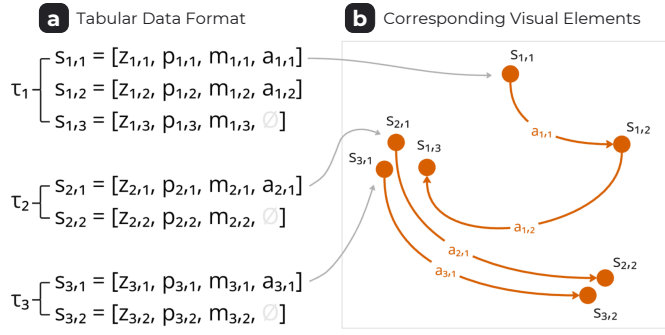


Fig. 2. **Data format and its visual interpretation.** (a) **Tabular Data Format:** Our input is a CSV-like table in which each row represents a state $s_{\tau,t}$ identified by a trajectory index τ and a within-trajectory step index t . Each row stores the projected 2D position $p_{\tau,t}$, the set of features chosen for computing the projection $z_{\tau,t}$, additional metadata $m_{\tau,t}$, and, where applicable, the action $a_{\tau,t}$ defining the transition to the successor state (with $\emptyset$ indicating that no successor/action exists for the final state). (b) **Corresponding Visual Elements:** Rows map to node marks positioned at $p_{\tau,t}$, and consecutive rows within the same τ are connected by directed edges representing the actions $a_{\tau,t}$, forming the embedded trajectories shown on the right.

defines the three data levels used throughout the workflow: states, transitions, and trajectories.

A. States

Each row or sample in the dataset corresponds to a state $s_{\tau,t} \in \mathbb{R}^d$, represented as a node in the visualization. A state is described by a high-dimensional feature vector, which we conceptually partition into the subset of features $z_{\tau,t}$ used to compute the projection and the remaining metadata $m_{\tau,t}$ that may still be used for visual mapping and search. The projection yields a two-dimensional coordinate $p_{\tau,t} \in \mathbb{R}^2$ that determines the node's position in the visualization. Each state is indexed by a pair (τ, t) , where τ denotes the between-trajectory index and t denotes the within-trajectory index when applicable.

Examples of state-only datasets include MNIST [6], Palmer Penguins [7], and Iris [8], where each state corresponds to one image, animal, or flower sample. In trajectory datasets, states instead occur as ordered instances within a larger sequence: in chess, each state corresponds to a board configuration between moves; in a robot simulation reinforcement learning environment, each state may correspond to a concrete scene configuration, such as the robot joint positions and angles.

The choice of projection method and projection features $z_{\tau,t}$ determines which high-dimensional relationships become spatial relationships in the map, enabling tasks centered on neighborhoods, clusters, and outliers. Metadata $m_{\tau,t}$ does not shape the map but may still support filtering, querying, and visual encoding.

B. Transitions

Transitions are defined between successive states. Each transition corresponds to an action $a_{\tau,t} = (s_{\tau,t}, s_{\tau,t+1})$, representing the transformation from one state to the next. Transitions are visualized as directed edges connecting the corresponding node positions in the projection. Using these

transitions, successor and predecessor relations are defined as $\text{next}(s_{\tau,t}) = s_{\tau,t+1}$ and $\text{prev}(s_{\tau,t}) = s_{\tau,t-1}$, whenever such states exist.

Examples of transitions include chess moves between board configurations (e.g., Nf4, knight to f4) or robot simulation actions between scene configurations (e.g., +0.1 rad at joint 2). More generally, transitions may encode domain-specific actions or temporal adjacency between consecutive states. In our data model, we capture each action together with its first state; potential additional action attributes can serve as part of the metadata $m_{\tau,t}$.

C. Trajectories

Trajectories are formed as ordered sequences of states connected through successor relations. All states sharing the same trajectory index τ belong to the same trajectory, while the within-trajectory index t determines their order along that path.

Continuing the previous examples, a chess trajectory corresponds to a complete game, composed of all board states and moves from the initial position to the outcome, and a dataset may contain many such games. Similarly, a robot simulation trajectory may correspond to one pick-and-place episode, capturing the sequence of scene configurations and control actions as the arm reaches for an object, grasps it, and moves it to a target location.

At the trajectory level, the main design implications concern representation and topology. Unlike graphs and networks, trajectories may visually intersect or overlap in the projection while each trajectory is linear, non-branching, and remains distinct from other trajectories even when states occupy the same projection position. We therefore encode trajectories in the previously mentioned simple tabular format, where each row represents one state, trajectory membership is given by τ , within-trajectory order by t , and the action to the successor state is stored with the source row. This lightweight representation makes existing tabular state-only datasets usable as-is, while still supporting trajectory datasets without additional structure.

III. RELATED WORK

We begin by situating projection enrichment and specifying the annotation terminology we adopt in relation to existing taxonomies. We then review the projection tools and workflows most closely aligned with our own. Finally, we discuss the degree-of-interest and visualization concepts that our method relies on.

A. Layout Enrichment and Annotation

Nonato and Aupetit surveyed the design space of multi-dimensional projections and identified two challenges: projections can be misleading due to neighborhood distortions, especially for non-linear methods, and scatterplot patterns alone are insufficient to understand high-dimensional data [3]. They suggest **layout enrichment** as the principal remedy for the second challenge, which our work aims to address.

Their taxonomy organizes enrichment by information *source*, *placement*, and *goal*. Our enrichment draws on both the *original* and the *visual space*, is *overlaid in place* on the scatterplot rather than in an *external panel*, and targets *content analysis* as a goal. By showing what the data looks like in context, it also helps reveal whether apparent clusters are meaningful or artifacts of distortion. It spans their *direct* (per-sample) and *cluster-based* (aggregate) pipelines, and adds difference annotations between sets that go beyond the enrichment types they catalog.

This layout enrichment is, more specifically, a form of **annotation**. Rahman et al. characterized annotations by the tasks they serve, the data they draw on, and the form they take [9]. In these terms, ours *summarize* and *identify* rather than merely highlight what is already shown. They are *internal* or *derived*, taken directly from the data or computed from it. They take the form of *text*, *enclosures*, and *glyphs*, attached to individual items, sets, and series, the last corresponding to trajectories in our setting.

Within these annotations, we distinguish lightweight **text labels** from pre-designed **detail visualizations** tailored to the data type: **single-item views** show one sample, **summary views** aggregate a set of samples [10], while **difference views** contrast two sets, either arbitrary or along a trajectory [11].

Because our annotations are realized as selections or queries rather than freeform geometry, they are *data-aware* in the sense of Heer and Shneiderman [12], allowing their references to be reapplied across visual encodings and filtering operations.

An automatically positioned annotation is an **inset**: Scalable Insets formulates inset placement as a cost optimization balancing detail, context, and locality [13]. We build on this placement for projection scatterplots, for instance accounting for point overlaps and positioning along cluster contours.

Annotation research has, however, largely targeted static, general-purpose charts [9], and annotations purpose-built for projections remain comparatively scarce. This gap motivates examining how existing projection tools handle annotation in practice.

B. Origins of Trajectory Projections

Before comparing annotation workflows more broadly, we first trace the projection lineage most directly tied to our data model: projections in which ordered states form trajectories.

The idea of representing trajectories in a projection originates in early computer-vision work: Pless treated a video as an image space produced by Isomap over its frames, with the temporal ordering tracing a trajectory through that space [14]. Time Curves fold a timeline so that similar moments lie close, revealing patterns of temporal evolution [15], and Projection Space Explorer extends this line by consolidating a taxonomy of the temporal patterns such trajectories exhibit [16]. Projections also serve to interpret reinforcement-learning agents: Zahavy et al. embedded DQN states with t-SNE to expose the structure of a learned policy and value function [17], a line continued by interactive tools such as VISITOR for multiscale exploration of agent state sequences [18].

TABLE I
TECHNICAL AND CONCEPTUAL WORKFLOW PROPERTY COMPARISON OF THE MOST RELATED INTERACTIVE PROJECTION TOOLS.

	PSE	Probing	Atlas	Splatters	TB Proj.	Ours
Technical properties						
Detail-view location	hover + side	hover + side	hover + side	in-place + side	in-place + side	in-place
In-plot cluster labels	IDs			in insets		
Annotates	P C D	P C D	P C D tabular	P C D	P C D	P C D
Trajectory data						
Image detail views						
Tabular / metadata views						
Custom domain views						
Automatic clustering		agglom.	density-based	parametric		HDBSCAN
Hierarchical annotation			coarse, tf-idf	parametric		
Placement strategy	—	—	map labeling	edge-aligned	—	annealing
In-tool (re)projection						
Custom-dataset upload						
Demonstrated scale (states)	4×10^4	$\sim 10^2$	$\sim 10^6$	5×10^4	3.5×10^4	4×10^4
Conceptual workflow properties						
Annotation structure source	manual	data-driven	data-driven	parametric	—	data-driven
Selection mechanisms	lasso	rect. + click	brush + query	lasso + click	click + query	lasso + query
Detail triggers	selection	hover + selection	auto + selection	selection	automatic	automatic
Hierarchy granularity control	—	manual	automatic	manual	—	automatic
Detail adapts to zoom		no zoom	labels only	no zoom		
Models user interest						
Serendipitous discovery	color	color	labels	exemplars		

● supported ○ partial ◯ absent — not applicable P C D point / cluster / difference

C. Annotation in Projection Tools

We now turn to the interactive projection tools most closely related to ours, including but not limited to trajectory-aware systems. They differ along several capability dimensions: where annotations are placed, what they can annotate (single points, clusters, or differences), how the user selects what to annotate, which data types their detail views support, whether they handle trajectory data, and whether clustering is automatic. Table I summarizes the technical and conceptual dimensions. Below, we highlight the most distinguishing trait of each tool.

Projection Space Explorer shows per-item detail in a fixed corner disconnected from the annotated point, and presents its summary and difference visualizations in side panels, all driven by manual selection, though it does support trajectory data [16]. Probing Projections shows detail through hover tooltips and a sidebar over clusters whose number the user sets by hand, and confines in-plot enrichment to heatmaps and error overlays [19]. Embedding Atlas clusters and labels automatically with zoom-stable text labels reminiscent of cartographic map labels, the closest analogue to our label layer, but delivers detail only as metadata charts in coordinated side views [20]. Concept Splatters comes closest to our visual

result, placing annotations beside contoured clusters in the plot, but its hierarchy is largely predefined, its inset placement is simpler than our optimization-based scheme, and its insets show fixed exemplars rather than aggregates [21]. TensorBoard Embedding Projector instead annotates every point with a thumbnail, which scales poorly and leads to overplotting, and it offers neither aggregation, an interest model, nor placement optimization [22].

None of these tools combines per-sample, cluster, and difference annotation with an automatically derived, data-driven hierarchy and placement in the context of the annotated data. They also differ conceptually, in whether structure is defined manually or automatically and whether detail is triggered by selection or shown without interaction. We defer the step-by-step comparison of these workflows on a common task to the evaluation (Section VII).

Across these tools, deciding which structures to annotate, at which level, and where to place each annotation stays a manual effort rather than something driven automatically by data structure and user intent, which is the mechanism we turn to next.

D. Degree of Interest and Visualization Concepts

A **degree of interest (DoI)** is a scalar value, conventionally between zero and one, expressing how interesting an element is to the user in the current analysis. Furnas’s original fisheye formulation combines an element’s a priori importance, such as its depth in a tree, with its distance from the user’s current focus [4], so a DoI is inherently a combination of signals. Van Ham and Perer apply this idea to large graphs, using degree of interest to support search, context, and expansion on demand [23]. We use this classical notion to define a **data DoI**: a per-element value from not interesting (0) to maximally interesting (1).

Our **layout DoI** instead scores how interesting each candidate pattern in the visualization is, blending the data DoI with the pattern’s salience in the layout, such as a cluster’s spatial footprint and stability, so that the most relevant patterns surface automatically. The layout DoI is not normalized to lie between zero and one, a departure from the classical DoI convention. We bound how many of the highest-scoring annotations appear at once with a **visual entity budget** [24].

Abello et al. give a modular definition of degree of interest for graphs, composed of specification, transformation, combination, and propagation components, with propagation assigning each element the maximum neighbor interest attenuated by distance [25]. We adopt their specification and propagation components, propagate our data DoI in the same distance-attenuated, neighborhood-maximum form, and merge several signals into the layout DoI through a combination component. We simplify their freely composable framework to a fixed composition suited to trajectory projections, and address the computational efficiency needed to propagate interest at interactive rates in Section IV.

Our method further builds on several established visualization concepts. *Multiscale visualization* presents data at different levels of abstraction depending on viewing scale, as in zoomable interfaces [26], [27]. *Semantic zoom* is one such technique, where the representation itself, not only its size, changes as the user zooms. A map, for instance, may swap a region’s name for its streets on zoom-in [26]. *Overview+detail* pairs a global overview with detailed views of selected parts, such as a thumbnail beside a zoomed-in view [28], while *focus+context* emphasizes a region of interest while keeping it embedded in its surroundings, as in a fisheye lens [4], [28]. We combine these concepts with degree of interest so that annotations resolve from cluster-level aggregates to per-sample detail across zoom levels surrounded by stable context.

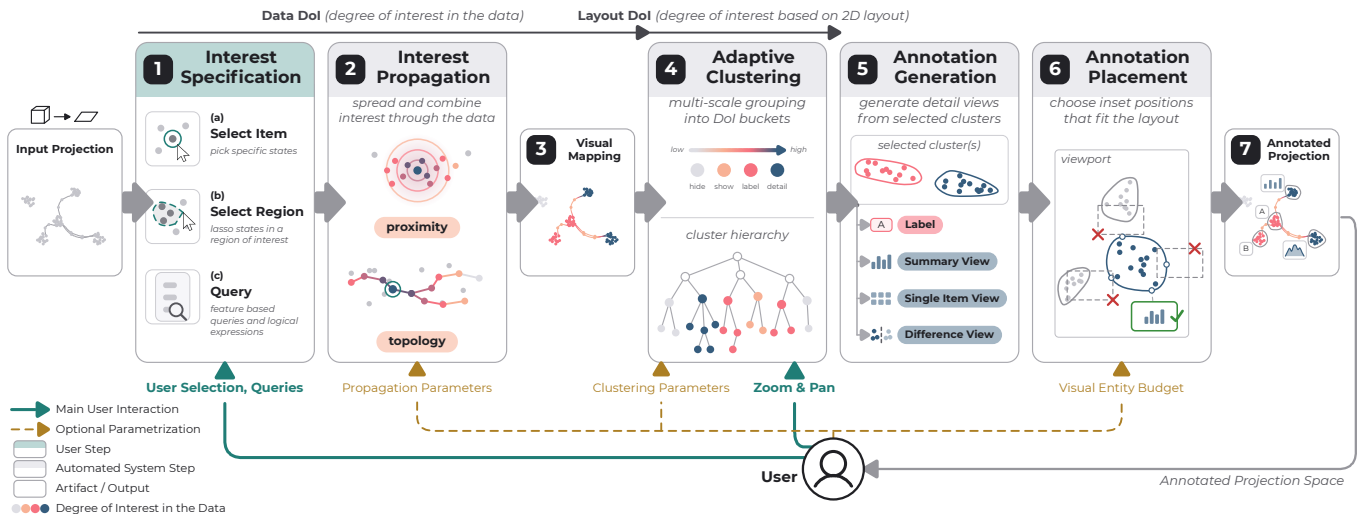


Fig. 3. **Workflow for understanding projection spaces through adaptive in-place details.** Starting from a fixed 2D projection, users (1) specify points of interest; (2) DoI propagation expands this focus to related states; and (3) visual mapping reflects the resulting relevance in the projection. (4) Adaptive clustering organizes and ranks relevant structures at the current scale, (5) annotation generation turns the selected structures into labels and detail insets, and (6) annotation placement integrates them into the available space. The result is (7) an annotated projection that continuously adapts to user interaction.

IV. METHOD

To support the interpretation of projection spaces through adaptive in-place annotations, we introduce a workflow that transforms a static projection into an interest-driven, whitespace-aware, annotated view (see Figure 3). Starting from high-dimensional data whose states or state groups can be meaningfully summarized and contrasted, we assume a fixed 2D projection that serves as the spatial substrate for exploration. On this basis, users *specify points of interest* (Section IV-A) through selections or queries, defining an initial degree of interest in the data. This focus is expanded via *DoI propagation* (Section IV-B), which spreads relevance across the projection based on spatial proximity and trajectory topology. The resulting data DoI also controls the visual mapping of nodes and edges, preserving the projection as context while emphasizing relevant elements. The data DoI then drives *adaptive clustering* (Section IV-D), which organizes relevant elements into a hierarchical structure for multiscale analysis and derives layout DoI scores for candidate clusters in the current view. *Annotation generation* (Section IV-E) applies a visual entity budget to select the candidates with the highest layout DoI and translates them into text labels, single-item views, summary views, and difference views. Finally, *annotation placement* (Section IV-F) positions the generated insets within the projection while avoiding conflicts with the surrounding layout. Together, these stages form a semi-automated loop: selections and queries execute the specification step, while pan and zoom update the view-dependent clustering. Each change triggers the subsequent stages that depend on it to recompute and update the visualization in real time. In the following, we cover each stage in more detail.

A. Interest Specification

Specification of interest describes the mechanisms by which users explicitly indicate which states should receive maximal initial data DoI before any propagation occurs. In the terminology of Furnas [4], these inputs define the *a posteriori* foci of the visualization: elements that are directly marked as relevant through interaction rather than inferred computationally.

We assume no prior knowledge about what is relevant to the user. Consequently, the *a priori* data DoI is initialized uniformly with $\text{DoI} = 1$ for all elements. This preserves the full visual context while placing annotations on the dataset's most dominant, structurally salient clusters for initial orientation. Once users begin specifying interest, the system re-enters the workflow with an updated DoI distribution shaped by prior interactions.

1) *Selection- and Query-Based Input*: The design space of such specification mechanisms is broad. A fundamental distinction can be drawn between *selection*-based and *query*-based inputs. Selection operates directly in the visual domain and includes interactions such as clicking individual states, sweeping a brush across a screen region, enclosing an arbitrary shape with a free-form lasso, or applying spatial lenses [29]. These correspond to single- and multi-focus specifications as discussed by Furnas [4].

Query-based specification operates in data space rather than screen space. Simple attribute filters can remove items that do not match feature constraints, while more expressive logical queries allow for compound conditions (e.g., selecting all chess states where $\text{material_balance} > 3$ AND $\text{next_move} = \text{Nf3}$). For trajectory data, query-based specification could also match action subsequences along paths, for instance, through regex-style formulations (e.g., $\text{^e4} \cdot \text{*e5\$}$).

Conceptually, searches for characteristic 2D trajectory patterns also fall into this category. Examples include stagnation, oscillation, or self-intersection discussed by Bach et al. [15], as well as patterns mentioned by Hinterreiter et al. [16], such as dense initial clusters diverging into multiple directions or bundles exhibiting differing trajectory flow directions.

2) *Hard and Soft Specification*: A further conceptual distinction can be drawn between *hard* and *soft* specification. In hard specification, membership is binary: a click or lasso either includes or excludes a state, and a filter predicate either matches or does not. In soft specification, also referred to as fuzzy or smooth selection [30], membership is graded, as with distance-weighted lenses, brushes with fuzzy boundaries, or score-based retrieval. Our system keeps the specification stage hard. Gradation is instead induced by the subsequent DoI propagation, which turns the binary selection into a graded interest distribution.

3) *Reduction to Three Modalities*: We reduce this design space to three fundamental modalities, illustrated in Figure 3(1): (a) single-item selection, (b) region selection, and (c) query-based selection. Together, they subsume the mechanisms above, providing broad expressive power through a simple interaction model, and define the seed set from which DoI is subsequently propagated.

B. Interest Propagation

Propagation turns the binary state specification into graded state interest between zero and one. It uses two complementary notions of relatedness: (i) proximity in the projection, as a proxy for similarity in the original feature space, and (ii) adjacency along trajectories, which captures continuity within an evolving process (see Figure 3(2)). The model below defines the data DoI.

Formally, let V denote the set of all states in the projection. For each state $x \in V$ we compute a Degree-of-Interest value $\text{DoI}(x) \in [0, 1]$, determined either directly by user specification or by the strongest propagation received from any other state. The propagation from state y to state x is the maximum of two channels, corresponding to the two structural relations present in our data (Figure 4).

1) *Projection Proximity*: Spatial distance in the 2D projection serves as a proxy for similarity in the original feature space, so interest spreads from a state to nearby states according to a monotone falloff with projection distance (see Figure 5). In practice, d_{emb} is typically the Euclidean distance in the projection plane. The selectable falloff is logarithmic by default, fading to zero within a bounded radius so low settings keep interest local rather than washing distant states with faint influence, like a soft cutoff radius. The parameter $w_{\text{prox}} \in [0, 1]$

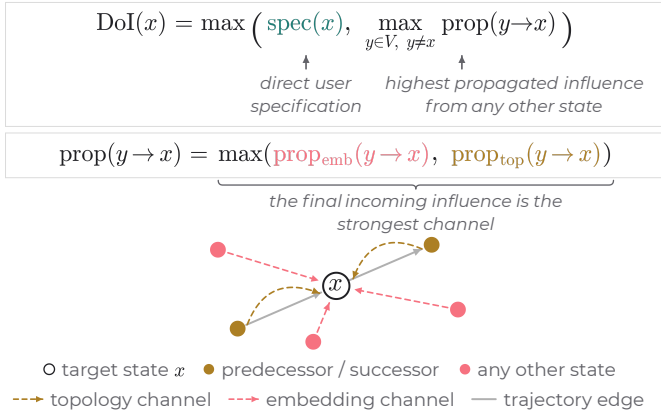


Fig. 4. State DoI and propagation-channel aggregation.

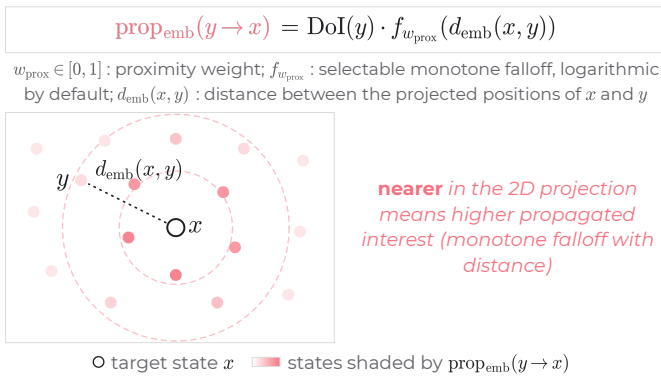


Fig. 5. Projection-proximity propagation.

controls the reach of proximity propagation. Setting it to zero disables the channel.

2) *Trajectory Topology*: Interest can flow forward and backward along paths, revealing coherent trajectory segments that may traverse distant regions of the projection space (see Figure 6). The parameters w_{forward} and w_{backward} control propagation strength along the two trajectory directions and how far the interest spreads along paths. For example, if $w_{\text{forward}} = 1$, interest propagates without decay through the entire forward trajectory. If $w_{\text{forward}} = 0.99$, a state located k steps ahead receives 0.99^k of the original interest value. Setting either parameter to zero disables propagation in the respective direction.

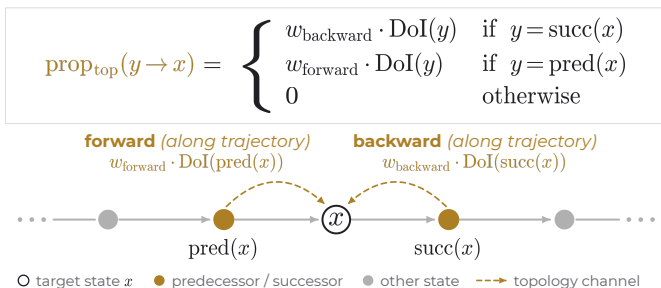


Fig. 6. Trajectory-topology propagation.

3) *Parametrization and Behavior*: Following the modular DoI framework of Abello et al. [25], whose specification and propagation components we adopt (see Section III), we expose exactly these three propagation strengths as interactive controls. Different settings yield distinct behaviors without requiring separate interaction tools: with all strengths near zero, DoI remains confined to the selected elements; strong forward or backward propagation emphasizes entire trajectory segments passing through the selection; combining topology with proximity highlights spatially separated yet topologically connected regions.

4) *Recursion and Convergence*: Propagation is recursive, since newly raised DoI values can trigger further propagation through both channels until convergence, and the proximity channel conceptually couples every pair of states. Contributions are combined by taking their maximum (Figure 4), corresponding to a logical OR: a state becomes interesting if any single relation provides strong evidence of relevance. As discussed in Section V, this max-based formulation allows the converged distribution to be computed at interactive rates.

5) *Edge Relevance*: In our current pipeline, DoI is propagated only over state nodes. Each edge then inherits relevance as the arithmetic mean of its predecessor and successor state DoI. We deliberately avoid direct edge-space propagation (e.g., from edge midpoints or tubular edge neighborhoods), because the projection encodes state similarity rather than edge similarity: distinct transitions may cross or have nearby midpoints despite connecting unrelated states. This design could be revised for layouts in which edges are embedded as first-class entities.

C. Visual Mapping of Nodes and Edges

The preceding steps yield a converged data DoI distribution that determines how nodes and edges are visually represented in the scatterplot. All elements remain spatially present whenever possible, while their data DoI controls their visual prominence [30] (see Figure 8). The implementation offers both opacity and a multi-hue mapping for the data DoI, with the latter used in the paper's figures for the sake of explanation. In practice, opacity is the default because color competes as a semantic channel for encoding data attributes, so we reserve color for these attributes and let opacity carry the data DoI. To support this use of color, the system recommends qualitative, sequential, or diverging palettes based on detected feature type and cardinality, warns when a qualitative palette provides too few distinct colors, and allows manual override of the type inference (for example, treating MNIST digit identifiers as categorical rather than sequential). Node size could also encode DoI, but we avoid this because larger markers in dense projections promote overplotting.

Trajectory edges are rendered as spline segments derived from Catmull–Rom interpolation. Because spline angles depend on the topologically connected neighbors, curved trajectories indicate more clearly where a path enters and leaves a shared dense region [31] than straight polylines, which become ambiguous where many lines meet [32]. Existing comparisons of curved and straight edges target uniformly dense graph

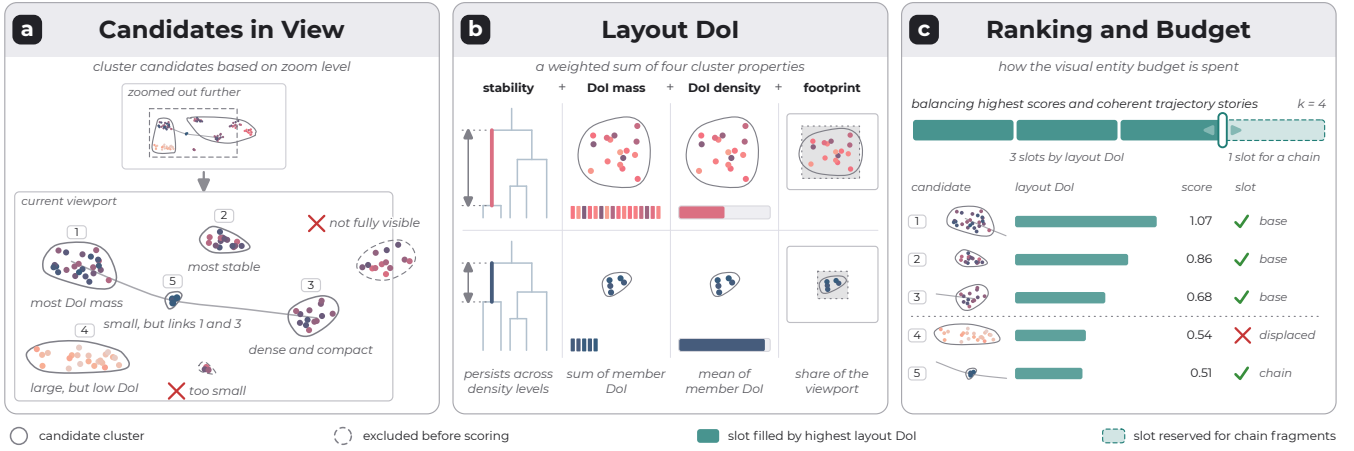


Fig. 7. **The layout DoI mechanism.** (a) Candidates surviving the zoom cut and the admission gates. (b) The four scored terms, each illustrated on a large cluster of mixed DoI (top row) and on the small, focused candidate 5 (bottom row). (c) The ranking and the budget of $k = 4$ slots: the top three go to the highest layout DoI, the fourth is reserved for a chain fragment. Candidate 4 is therefore displaced by candidate 5, which outranks it only along the trajectory through clusters 1-5-3. Scores are illustrative.

layouts [33]; in our trajectory setting, we found splines easier to follow, particularly for identifying branch structures and trajectory flow through dense regions.

D. Adaptive Clustering

Three thresholds applied to the propagated data DoI partition states into four relevance buckets, reflecting the four levels of emphasis our design distinguishes: faded, transparent, label-eligible, and detail-eligible (see Figure 3(3)). The two lower buckets are retained as context only. The upper two mark states relevant enough to be described in their own right, and they form the input to the clustering described in this section: only these states are grouped into the hierarchy, ranked, and eventually annotated.

Which of the label- and detail-eligible states actually receive an annotation further depends on the layout DoI, which scores clusters in the current viewport. Adaptive clustering provides the multiscale structure through which the layout DoI transfers annotations across levels of aggregation as the view changes. A central design decision is to generate these levels dynamically rather than constrain the visualization to a predefined hierarchy, allowing it to adapt to the granularity required by the current view. Conceptually, any method that discovers such a hierarchical structure could fill this role. We realize it based on density-based, hierarchical clustering.

1) *Hierarchical Clustering*: The choice of clustering methodology is guided by two practical constraints: (1) interactive analysis should remain parameter-light, avoiding methods that require a fixed number of clusters; and (2) the cluster hierarchy must remain consistent during multiscale drill-down under pan and zoom. Among the methods satisfying these conditions, DBSCAN [34] enables near-instant viewport-local recomputation but does not preserve cluster hierarchy across zoom levels. We therefore use the costlier HDBSCAN [35] for stable multiscale navigation. DBSCAN remains a potential future fallback when faster approximate recomputation is preferred.

One hierarchy is computed per selection state, over exactly the label- and detail-eligible states of the current DoI distribution, and is then reused across all zoom levels until the data DoI changes.

2) *Cluster Ranking*: We rank clusters in the hierarchy to later determine which should receive annotations given the limited screen space and human perceptual limitations. On every viewport change, cluster ranking is therefore updated. Because no single criterion (area, density, point count, or a dendrogram cut) is sufficient, we use a hybrid, viewport-aware semantic zoom pipeline (see Figure 7).

Candidates are obtained from a zoom cut that replaces a cluster with its children when its projection footprint exceeds an upper bound, a configurable fraction of the viewport, it is no longer fully visible, or its children are separated by a clear screen-space gap, revealing visually distinct but small subclusters. Therefore, zooming in splits clusters into their children, and zooming out closes the gap and re-merges them.

Candidates that are fully visible and whose footprint clears a lower bound, also a configurable fraction of the viewport, are ranked by their layout DoI, a weighted sum of HDBSCAN-cluster-stability, total member data DoI, mean member data DoI, and projection footprint. Stability and data DoI mass are normalized across the current candidates, while footprint is measured as a bounded fraction of the viewport. Mean member data DoI is independent of cluster size, so once a selection concentrates interest in part of the data, small but strongly focused clusters can compete with larger ones.

For trajectory data, we additionally maintain a *trajectory continuity ranking*: a separate ranking of candidates whose layout DoI may be too low otherwise but whose annotation would extend a coherent sequence along a selected trajectory. Here, candidates rank high when they share trajectories with high-layout-DoI clusters that pass through them, have high mean member DoI, and offer sufficient surrounding white-space.

3) *Manual Override*: Because regions of interest may not always align with density-based clusters, users can directly

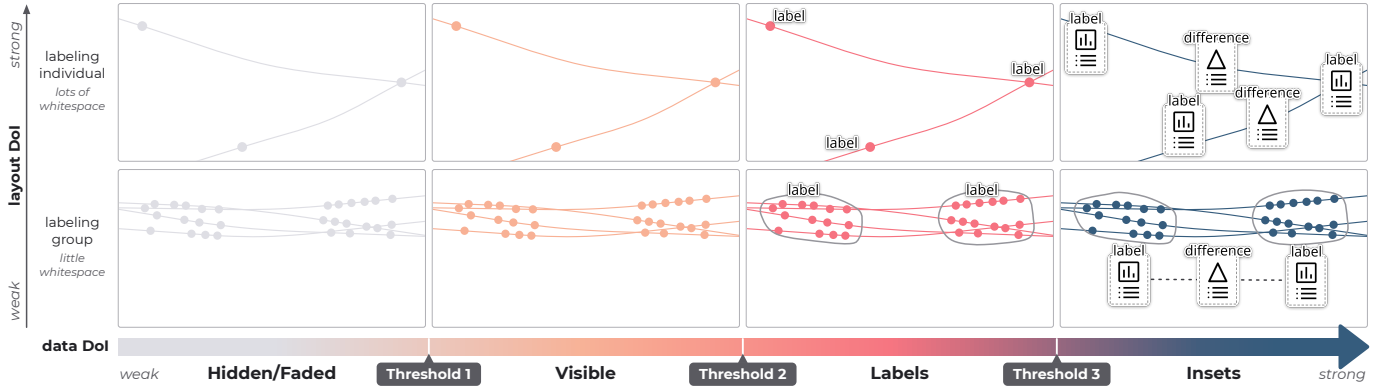


Fig. 8. **Visual Representations based on DoI.** Level of expressiveness follows data DoI, whereas annotation granularity follows layout DoI. With increasing interest, elements are shown fully opaque, receive annotations, and are augmented by insets with detail views.

lasso states for rendering a detail visualization. This provides a simple, manual override for the layout DoI in the cluster hierarchy when the automatic mechanism does not capture the desired structure.

4) *Resulting Multiscale Analysis:* Using a set of defaults that were tuned once, the hierarchy, viewport-dependent cut, candidate ranking, and trajectory continuity ranking together constitute the layout DoI mechanism that transfers annotations from aggregate clusters to their children and eventually to individual states under semantic zoom. Cluster heterogeneity remains visible in the persistent scatterplot context: distinct dense regions separate into individual subclusters upon zooming. Inset designs may additionally encode any data-derived summary of within-cluster variation.

E. Annotation Generation

This stage turns the ranked candidates into the single-item, summary, and difference visualizations, or labels that appear in the view. A visual entity budget first cuts the ranking down to what can be shown. The content of each annotation is then generated from the underlying data, and cluster membership is communicated by contours and leader lines.

1) *Visual Entity Budget:* Two visual entity budgets cap the annotations shown at once. The cluster budget caps active cluster annotations. An optional balance divides its k slots between the most salient clusters, identified by high layout DoI, and coherent, story-like annotation along trajectories, supported by the *trajectory continuity ranking*. A separate edge inset budget caps difference insets. Hovering bypasses this cap for the hovered cluster's comparisons. Flicker near the cluster-budget boundary is reduced via hysteresis with separate activation and deactivation thresholds. The optimizer could also vary these budgets, potentially improving the converged static view. However, avoiding insets repeatedly appearing and disappearing would require optimizing an off-screen model to convergence before presenting the result, which would be a departure from our seamless workflow.

2) *Annotation Representations:* Text labels communicate compact semantics at low visual cost, while detail insets expose higher-dimensional structure not visible in the 2D projection. As summarized in Figure 8, data DoI determines

the representation layer, while available whitespace determines whether annotations describe clusters or individual states. When many transitions directly connect two clusters represented by summary insets, the insets are linked by a straight line with a centered difference inset explaining the transitions between them. Hovering a difference inset highlights the corresponding transition bundles and their start and end states (Figure 10(6)). Hovering a summary inset instead reveals differences to all other summary insets regardless of connection strength (Figure 10(5)).

3) *Generation From Data:* Annotation content is generated from the underlying data rather than fixed to predefined concepts (i.e., labels or hierarchies). Systems that rely on fixed labels support navigation and progressive disclosure only for information already encoded in them, whereas dynamically generated detail insets expose the high-dimensional state representation, thereby enabling inspection along arbitrary dimensions.

Text labels, if a feature is chosen as such or generated from the data, are used directly for individual elements and aggregated for clusters. For flat categorical labels, we use the majority label and append “+” when the cluster contains a substantial mixed remainder. Hierarchical labels instead collapse to their shared level, such as *Queen’s Gambit: ** for *Queen’s Gambit: Accepted* and *Queen’s Gambit: Declined*. We also support TF-IDF term scoring by treating each cluster as a document, which favors terms characteristic of that cluster among the other selected clusters rather than merely frequent across the dataset.

Inset design depends on the data domain. Grid-structured states preserve their natural arrangement: For example, summaries of clusters in datasets of chess games or Rubik’s cube solutions show the most frequent element per cell, with frequency encoded by opacity or size. Image clusters are summarized by pixel-wise averages, keeping static regions sharp while moving objects blur across their motion envelope. Single-item insets show the original sample, while difference insets compare the sets of states that directly connect two clusters. For instance, for images, a blue–orange diverging scale distinguishes removed from added content.

For tabular data, each feature is represented by a smoothed

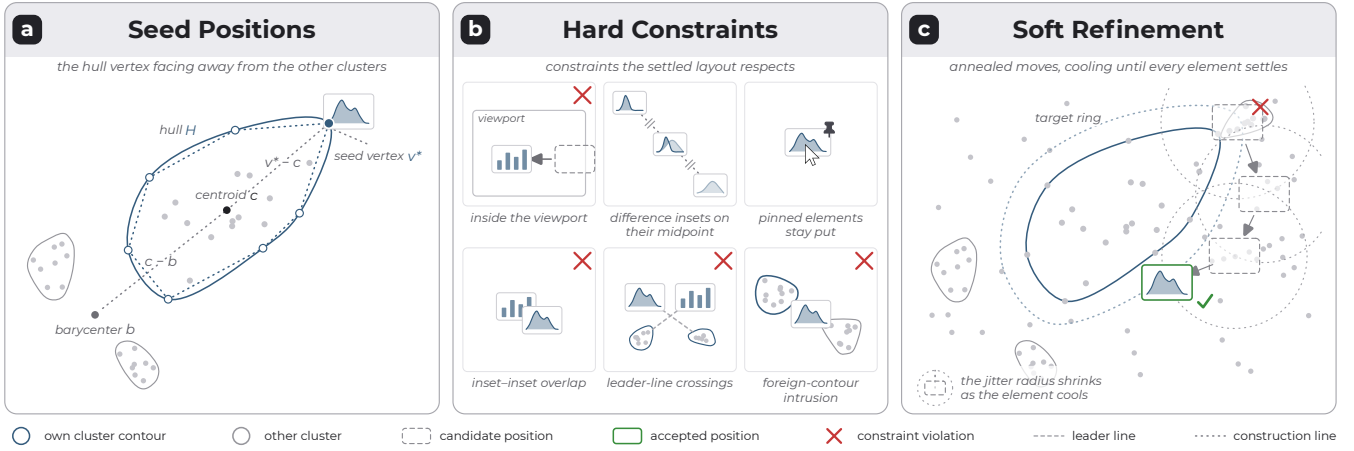


Fig. 9. **Annotation placement.** (a) An annotation is seeded facing away from the other clusters, offset outward and away from the centroid. (b) The constraints a settled layout respects, with crosses marking rejected configurations. (c) The candidate is moved by simulated annealing, its jitter radius shrinking as the element cools, until it settles and avoids violating constraints.

density curve over a histogram, following prior work [11], [19]. Features are ordered by importance, defined as how strongly their distribution distinguishes the cluster from its reference: the full dataset for summaries and the paired cluster for differences. Rather than the previously used χ^2 score [11], we use Jensen–Shannon divergence because it is bounded and independent of sample size, keeping scores comparable across features and clusters of different sizes. Finally, bounded power-law mappings scale node and difference insets with cluster size, improving readability without allowing individual annotations to dominate the view.

4) *Membership and Leader Lines*: Cluster membership is communicated by a single outer contour per active cluster, offset from the member geometry, drawn as a smoothed closed boundary with an outline-under-stroke treatment that keeps it distinguishable over dense backgrounds. Alternatives such as star-shaped centroid links or stacked density contours [11], [36] interact poorly with our setting, since they add further edge-like marks on top of already dense trajectory geometry. Annotations are most readable near their own contour, so insets seek positions on it. Whenever an inset comes to rest away from its cluster, a leader line re-establishes the association, styled with the same outline-under-stroke treatment and an optional dash pattern to separate it from trajectory splines.

F. Annotation Placement

This final stage completes the workflow by turning the generated annotations into a concrete layout: candidate positions are seeded, hard layout rules exclude invalid configurations, and an optimization refines additional soft criteria such as source proximity and temporal stability (see Figure 9). In contrast to side-panel detail-on-demand designs, we deliberately target superimposed, *in situ* annotations: they support navigational context, exploit the Gestalt principle of proximity, reduce interaction overhead during iterative exploration (see Section VII-A), and bring into the interactive setting the *in situ* enrichment that authors of static projection figures apply by hand (Section III).

1) *Simulated Annealing*: We adapt the simulated-annealing placement of Scalable Insets [13] to projection scatterplots. At each step, the optimizer selects one movable cluster annotation, proposes a positional jitter, and evaluates the resulting global cost. Lower-cost moves are accepted, while higher-cost moves may be accepted according to the selected element’s temperature. Each element cools independently, progressively reducing its move radius as its placement stabilizes.

We adapt the formulation by using cluster contours as source geometry, accounting for foreign contours and leader-line crossings, and positioning difference insets through their associated source elements rather than annealing them independently. Each cluster inset seeks a position near the exterior of its own contour, so that gaze travels directly from the cluster to its detail view. The automatic result remains manually adjustable: insets can be dragged, repositioned, and pinned. Pinned elements are excluded from subsequent optimization until released. A direct cartographic mode bypasses the optimization entirely and leaves each annotation at its source location.

2) *Seed Positions*: Let c be the centroid of a cluster, H its convex hull, and b the barycenter of the other clusters. We first identify the hull vertex facing away from the cluster’s peers, $v^* = \arg \max_{v \in H} (v - c) \cdot (c - b)$. Here, v denotes a candidate 2D hull vertex and v^* the selected vertex: among all hull vertices, we choose the position that lies farthest in the direction pointing away from the other clusters. The corresponding annotation is then seeded just outside its own contour, as shown in Figure 9(a).

The system remembers the last settled world position associated with each inset membership. When the same membership reappears after navigation, that position is restored and remains fixed. A membership change instead updates the source centroid, recomputes the seed, and reactivates the element for placement.

Difference insets follow their associated source geometry rather than being optimized. Those that compare individual states are placed at the midpoint of their connecting edges.

Difference insets between clusters are pinned collinearly between the two compared cluster insets.

3) *Hard Constraints*: Placement follows two tiers of constraints. Strict rules keep annotations inside the viewport, preserve the positions of settled elements while their layout remains valid, and maintain the prescribed relationships of difference insets to their source elements.

Inset-to-inset overlap, leader-line crossings, and intrusion into unrelated cluster contours form a second tier. These configurations are not forbidden outright during the search, but incur severe costs that act as de facto hard rules. Violations may therefore occur temporarily during refinement, but are penalized so heavily that they are effectively excluded from the settled layout. All constraints are shown schematically in Figure 9(b).

4) *Soft Refinement*: Within the valid placement space, the cost model balances additional factors such as distance from the source contour, movement from the previous position, and occlusion of source elements or projected data.

With our default configuration, the search keeps annotations close to their source contours while preserving viewport containment, difference relationships, and positions that remain valid across view changes. Figure 9(c) traces the clockwise movement of an inset along its cluster as it approaches its ideal position.

G. Resulting Annotated Projection

The final result of the complete workflow is a semi-automated, continuously updated annotated projection in which context is preserved, relevant structure is emphasized, and detailed evidence appears where and when it is needed with minimal manual curation. As users pan and zoom, the active cluster set changes, and the system updates annotation granularity, content, and placement in real time. Existing placements are retained whenever their memberships and constraints remain valid, while newly appearing or changed elements are seeded and optimized. Respecifying interest reruns the complete workflow.

V. IMPLEMENTATION

Our prototype is a web-based TypeScript/React application, deployed as a static single-page app with no server-side computation at runtime. This does not restrict users to the example datasets, as they can simply load their own via drag-and-drop, or by passing an in-memory data frame to our Jupyter widget, which renders the full system inline in a notebook after a pip install, with no server component.

A. Design Considerations

We derived our architecture by asking two questions of each visual element. *Is it on the per-frame hot path?* Nodes, spline edges, and arrowheads are updated on every interaction frame and therefore rendered with WebGL2, batched into one draw call per element type and gated by a dirty flag that allows at most one redraw per animation frame. *Is the bottleneck human perception rather than computational performance?*

Since the visual entity budget limits how many annotations are visible at once, insets can be ordinary React components rendered in the DOM. Supporting a new dataset then requires a single new component instead of shader code. Between these extremes, blocking computation runs in Web Workers with epoch-based cancellation, which discards stale jobs before they can block the interface. Everything that can be derived in advance is precomputed offline in Python and shipped with the dataset. This includes the most common view, the full-context overview with $\text{DoI} = 1$ everywhere, which makes opening a dataset and resetting a selection require no recomputation instead of being the most expensive operations.

Many further parts of the pipeline were optimized in similar ways. In the following, we detail the two problems whose solutions we consider most relevant beyond our prototype: propagating interest through the projection and updating the inset set under semantic zoom.

B. Propagating Interest Through a Fully Connected Space

The propagation model in Section IV-B combines spatial proximity in the 2D embedding with predecessor and successor propagation along trajectories (Figure 4). Direct evaluation is costly for two reasons. Spatial influence couples all pairs of states, requiring $O(n^2)$ work per update step, and the two channels feed each other recursively, requiring iteration to convergence. With maximum aggregation and monotone spatial falloff, however, the spatial influence reaching a state in any round depends only on its distance to the nearest source. We therefore reformulate each round of spatial propagation as a distance-to-nearest-source field over the projection.

The projection extent in data coordinates is rasterized to a fixed grid of square cells, with 1024 cells along the longer axis, independent of zoom. The first round seeds the grid with the selected states and computes an exact Euclidean distance transform in $O(C)$ for C grid cells [37]. States obtain distances by bilinear sampling. Rasterization and read-back are $O(n)$, so a full update is $O(n + C)$, with C fixed by grid resolution and projection aspect ratio rather than by n . One forward and one backward pass along each trajectory then propagate topology with per-hop attenuation in $O(n)$ total. The two channels then alternate. States raised by the trajectory passes spread their interest onward spatially, weakened according to their already-reduced value, and each new round recomputes the field from these many sources with a chamfer sweep, which closely approximates Euclidean distance at the same $O(C)$ cost. Each round can only raise interest, and every additional hop contributes less, so the alternation converges quickly, after at most 10 rounds across all benchmarks (Section VII-B). During dragging, the same alternation runs in single precision on the GPU, updating the view with every slider movement, while a background worker concurrently computes the exact field and streams it in behind the preview. Every release commits the exact result, so the values that persist always come from the full-precision engine.

C. Scalable Inset Updates Under Semantic Zoom

The general problem is maintaining a set of annotation insets over a labeled spatial layout as the view changes: which

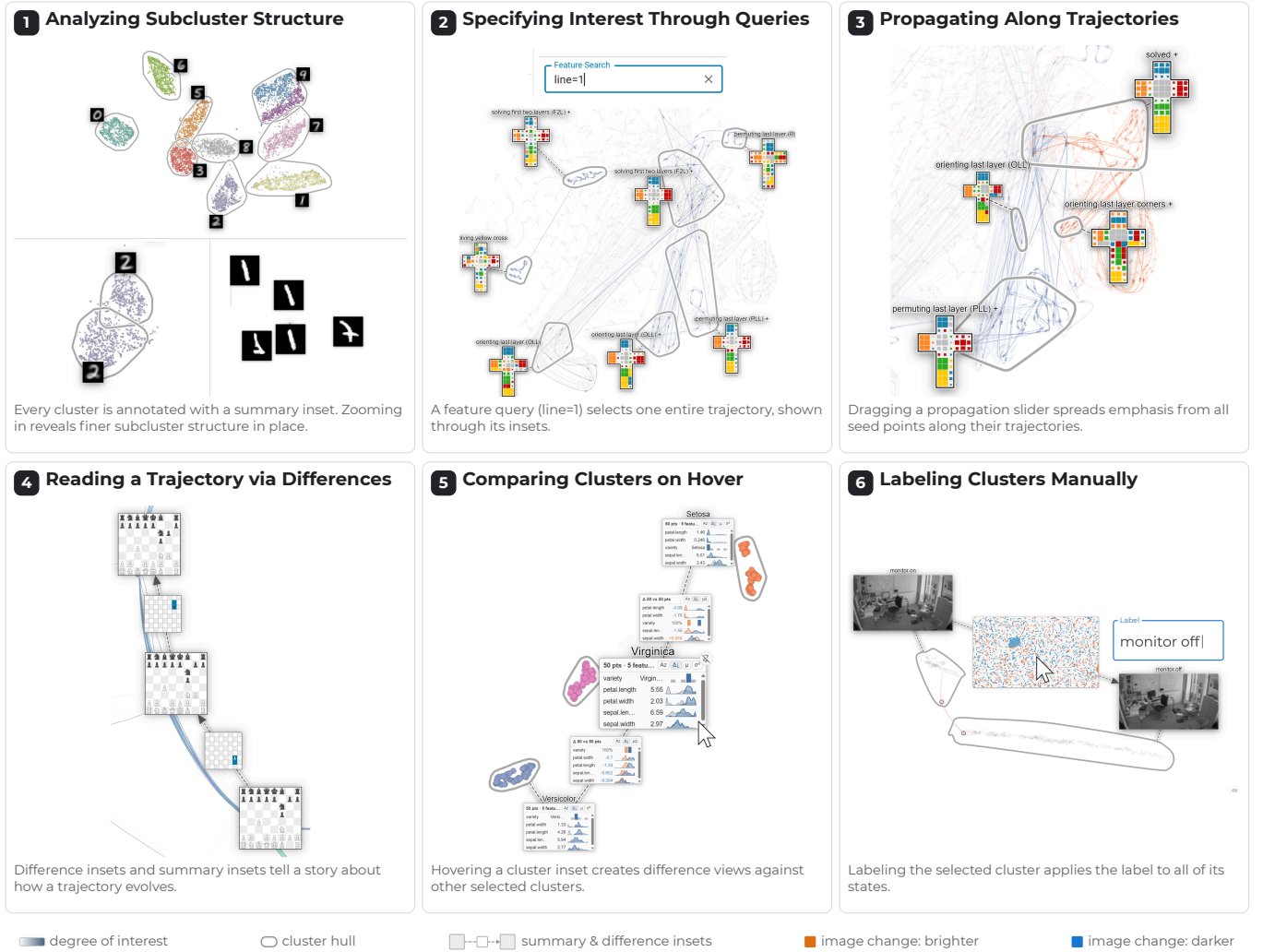


Fig. 10. Usage scenarios: (1) MNIST overview and zoom-in [↗](#); (2) Rubik’s cube trajectory query [↗](#); (3) backward trajectory propagation [↗](#); (4) chess opening through summary and difference insets [↗](#); (5) Iris cluster comparison on hover [↗](#); (6) CCTV cluster labeling [↗](#). Table II summarizes the datasets excluding iris.

regions deserve an inset must be re-decided, and each inset must be placed adjacent to its region without overlapping other insets or occluding data, smoothly, on every pan and zoom.

The first part, deciding *which* insets exist, is a question of granularity. The density-based cluster hierarchy is computed once and encodes all granularities. Each viewport change triggers re-deriving which of its clusters are active, following our layout-DoI activation criteria. This must happen without stalling the frame: a single $O(n)$ traversal applies our cluster split triggers to the hierarchy, and a result cache returns the previous cut in $O(1)$ whenever viewport and configuration are unchanged. Clustering itself is recomputed only when the selection changes, off the main thread in a Web Worker with epoch-based cancellation.

The exact *placement* problem is NP-hard [38], so we adapt the simulated-annealing implementation of Scalable Insets [13]. Its cost model remains fully configurable at runtime, including terms for data occlusion and mental-map stability. However, we found a minimal default objective sufficient. It

comprises two soft terms, distance to the target contour and inset-overlap pressure, both subordinate to near-lexicographic penalties against inset overlap, leader-line crossings, and intrusion into foreign contours. Each inset cools independently and the optimizer sleeps once all have settled. It pauses during zoom, selectively reheats only disturbed insets, and restores remembered positions when clusters reappear.

VI. USAGE SCENARIOS

We demonstrate six capabilities across five datasets, each in a view chosen to make the resulting effect directly visible. Phrases marked with [↗](#) link to live demos recreating the corresponding panels.

Analyzing Subcluster Structure: The MNIST [6] projection illustrates progressive disclosure through navigation (Figure 10(1)). The overview identifies the digit classes through summary insets. Zooming into the “2”s resolves them into subclusters with and without a lower-left loop, while the elongated “1” cluster separates by stroke angle. Further zooming

exposes individual outliers at its fringe, including “7”s and J-shaped “1”s. Structure is thus revealed in place and at the granularity demanded by the current view, without selection or side-panel comparison.

Specifying Interest Through Queries: In the Rubik’s cube projection, the query `line=1` selects one complete solution trajectory and annotates the regions it traverses (Figure 10(2)).

Propagating Along Trajectories: In the Rubik’s cube projection, selecting the solved states marks the common endpoints of all solution trajectories (Figure 10(3)). Increasing backward propagation then spreads interest from these seeds through their predecessors, revealing the regions and paths that lead toward solving the last layer. The propagation is previewed continuously during dragging, with annotations updated on release.

Reading a Trajectory via Differences: In the chess projection, summary insets along an opening bundle show successive board-state clusters, while difference insets reveal the moves between them (Figure 10(4)). Together, they recount how the opening evolves directly along its path through the projection. The same encoding also reveals recurring structure: zooming into a loop in the Rubik’s cube projection exposes the repeated move sequence $R \ U \ R' \ U'$, a beginner-method corner-solving cycle (Figure 1(2)).

Comparing Clusters on Hover: The user first drags a local `iris.csv` file of Iris flowers [8] into the application and projects it with UMAP using the built-in projection mechanism. Hovering a cluster inset then compares it with every other selected cluster through synthetic pairwise difference views. In the resulting Iris projection, hovering the central *virginica* cluster reveals how its feature distributions differ from the selected *setosa* and *versicolor* clusters (Figure 10(5)).

Labeling Clusters Manually: Cluster insets also support manual batch labeling: selecting one opens a text field, and submitting a label applies it to every member state. In the Edinburgh office CCTV projection [39], zooming splits an apparent empty-room cluster into two subclusters. Their difference inset highlights the monitor region, revealing that the split corresponds to the monitor switching off. The user can therefore label the two verified subclusters “monitor on” and “monitor off”, assigning each label to many frames at once rather than frame by frame (Figure 10(6)).

VII. EVALUATION

We evaluate our approach in two parts: a **qualitative results inspection** (Section VII-A) of the insights and interaction effort supported by representative annotated-projection methods, and a **runtime evaluation** (Section VII-B) of all workflow stages across our datasets.

1) CCTV Clusters:

A. Qualitative Results Inspection

We identify insights communicated by representative annotated projections, map them to the projection-analysis tasks of Nonato and Aupetit [3], and examine whether and how each method reaches them. For evaluating workflow efficiency,

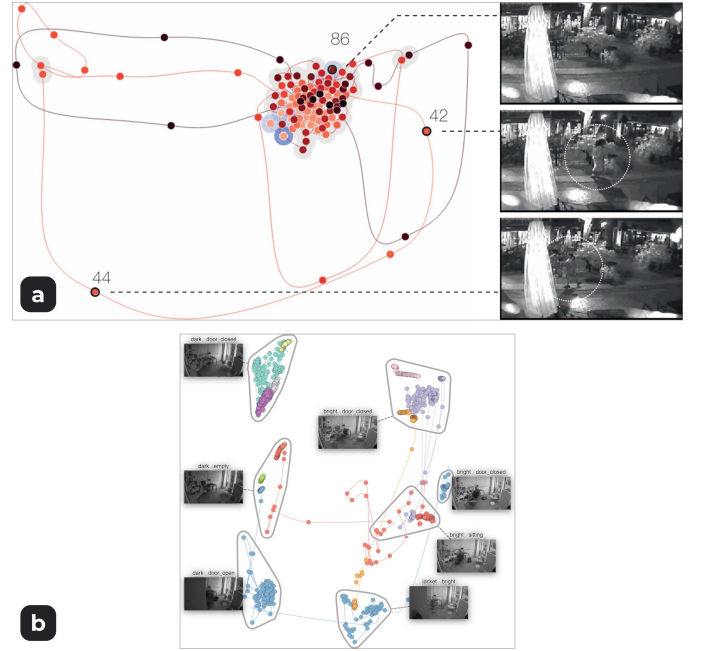


Fig. 11. (a) Manually annotated Time Curves view [15]; (b) our automatically annotated projection of the Edinburgh office CCTV data [39], produced without user intervention.

we additionally count minimal user actions, including brush, zoom, pan, click or menu choice, typed query, connection drag, and pane opening. View switches and unsuccessful tasks are reported qualitatively.

Bach et al. use manually annotated Time Curves of surveillance video to reveal static scenes, motion patterns, and unusual events [15]. This supports the task of *discovering and naming clusters*, such as empty-room scenes or people moving through the room. As the original data was unavailable, we constructed an equivalent trajectory projection from multiple days of the Edinburgh office CCTV dataset [40], representing frames through an autoencoder and PCA.

Figure 11 contrasts the manually enriched Time Curves view with the initial view of our automatically annotated projection. Aggregate insets reveal both cluster meaning and internal variation, including lighting changes along the horizontal axis, door states and whether a jacket is hanging from it vertically, dense sitting or empty-room scenes, and paths corresponding to recurring movement and scene changes. Our method therefore reproduces the task of *discovering and naming clusters* directly in the projection and without any additional user interaction. Because the reference annotations were added by the authors and not produced through an interactive workflow, no interaction count is meaningful for this comparison.

1) Fashion-MNIST Subclusters: For Fashion-MNIST [41], we define task T1 to identify the shoe-related clusters and task T2 to find the flip-flops within the sandals cluster. The flip-flop subcluster is one illustrative instance of a broader class of analysis targets not encoded by dataset labels or predefined hierarchies. Analogous targets could include fishnet tops or asymmetrical cutout dresses in Fashion-MNIST, looped 2s or anti-diagonal 1s in MNIST, or any other characteristic that

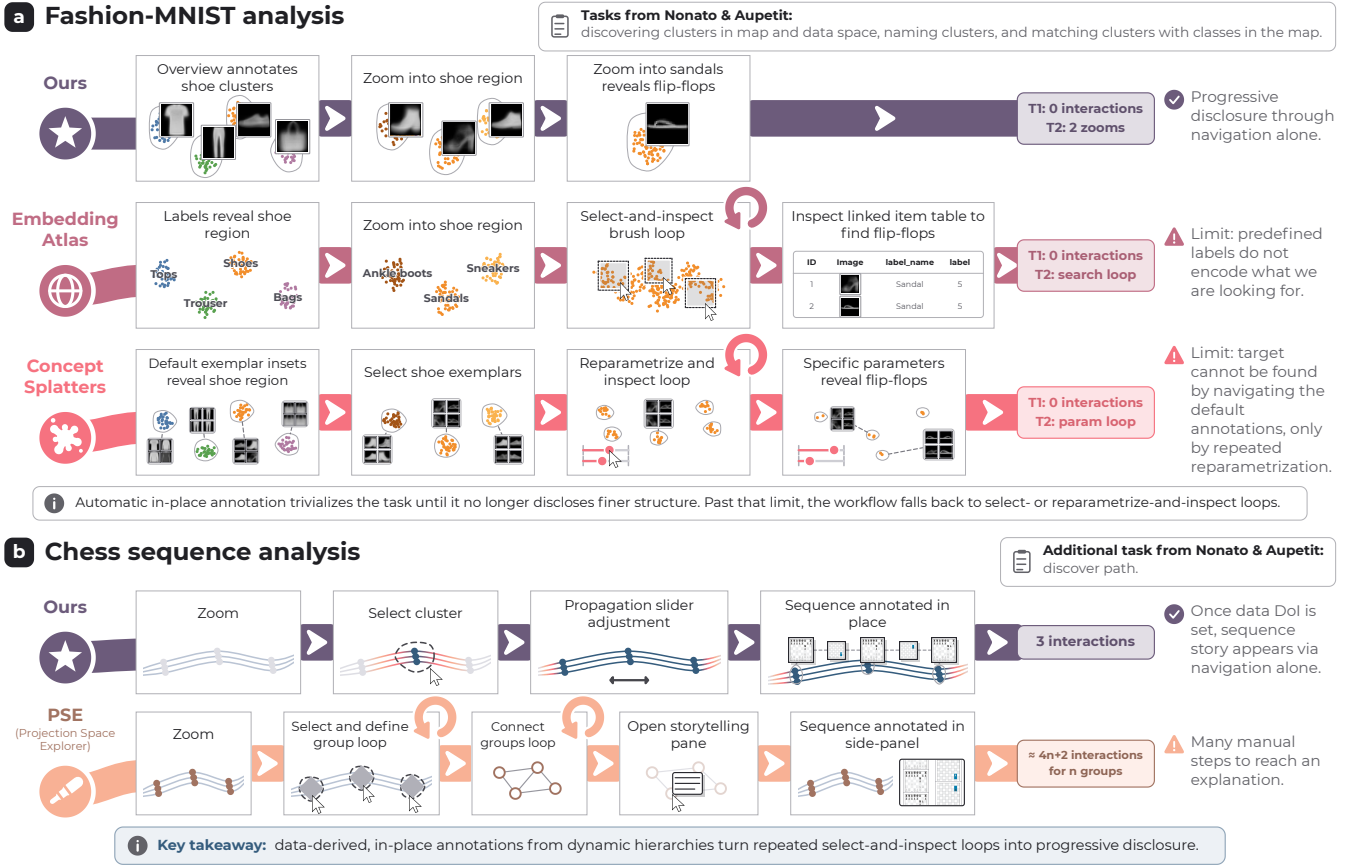


Fig. 12. **Interaction workflows for the two comparisons.** Each row traces one tool from the initial view to the resulting explanation. Boxes schematically show successive interface states, circular arrows mark repeated loops, and the final boxes summarize the interaction cost. Panel (a) compares Fashion-MNIST cluster analysis; panel (b) compares chess-sequence analysis.

users seek to discover in the high-dimensional data instead of retrieving from existing labels. T1 and T2 instantiate *discover clusters in map and data space, name clusters, and match clusters with classes in the map* [3]. We compare against Embedding Atlas [20] and Concept Splatters [21], which likewise annotate the overview in place (see Figure 12(a)).

All three approaches allow users to complete T1 without interaction: their annotations expose the shoe region, which can then be explored through navigation. In our tool, two zooms progressively reveal the shoe classes, sandals, and finally the flip-flop subcluster. Embedding Atlas supports this workflow only while the sought distinction is pre-defined as dataset labels and exposed at the available TF-IDF hierarchy granularity. Once its labels stop at sandals, T2 becomes a repeated brush-and-inspect search in the linked item table. Concept Splatters similarly supports navigation through exemplar insets and a largely predefined hierarchy, but T2 becomes a repeated reparameterize-and-inspect loop in which users adjust the level-of-detail bandwidth and threshold sliders until the flip-flop subcluster emerges.

We exclude TensorBoard Embedding Projector [22], whose unaggregated thumbnails turn T2 into visual search, and Probing Projections [19], which adds no further mechanisms beyond those already evaluated, as evident from Table I.

2) *Chess Sequences*: Projection Space Explorer characterizes chess trajectories through manually defined and connected groups whose board summaries and move differences appear in a storytelling pane [11], [16]. This supports the trajectory-oriented task of *discovering a path in data space*: identifying recurring positions and the moves connecting them.

Figure 12(b) compares both methods on the same data and sequence. To isolate explanation from search, we assume the target game bundle is known; otherwise, Projection Space Explorer would begin with a select-and-inspect loop. Our workflow requires three interactions: one zoom, one selection, and one propagation-slider adjustment to configure trajectory selection. Selecting the bundle then reveals its recurring positions, moves, and branches directly in the projection. Projection Space Explorer reaches the same explanation through $4n + 2$ interactions for n groups, or 14 for the three-group sequence shown: zooming, lassoing and defining each cluster as a group, connecting the groups, and opening the storytelling pane through the context menu.

Across these comparisons, predefined annotations and hierarchies provide efficient navigation while they encode the required distinction, but revert to select-and-inspect loops or fail once their available detail is exhausted. **Data-derived, in-place annotations from dynamic hierarchies instead turn repeated selection and inspection into progressive**

TABLE II

EVALUATION DATASETS AND MEDIAN RUNTIME BY WORKFLOW STAGE. QUERY SEARCH SPANS FROM SUBMITTING A TEXT QUERY TO THE FULLY PROPAGATED VIEW. PLACEMENT REPORTS THE SETTLING TIME OF ITS NON-BLOCKING REFINEMENT (6 MS PER FRAME) AFTER ZOOMING, WITH ACTIVE INSET COUNTS IN PARENTHESES.

Dataset	States	Traj.	Segments	Per-state content	Specification		Propagation	Adaptive clustering		Visual mapping
					Slider drag [fps]	Query search [ms]	DoI commit† [ms]	Zoom cut [ms]	Inset placement [ms]	
CCTV	5,311	10	106,020	image, ~9k px	162	50	70	1.1		2,736 (7)
MNIST	10,000	—	—	image, 28×28	138	68	67	1.8		3,334 (6)
Fashion-MNIST	10,000	—	—	image, 28×28	120	76	75	1.8		3,668 (5)
Rubik's (large)	21,414	200	424,280	6×3×3 grid	110	81	79	2.8		1,638 (7)
Chess	34,580	3,397	623,660	board, 8×8	150	91	89	2.7		3,021 (7)

Segments = individual spline segments — not applicable (no trajectories) † build + convergence, once per new selection or query; a slider release re-runs only convergence (n) = active insets

disclosure at the granularity required by the current view.

B. Runtime Performance by Workflow Stage

We benchmarked each runtime-critical workflow stage (Table II). For specification, we measured the preview frame rate during continuous propagation-slider dragging and the latency of a text-query commit. The remaining stages cover the DoI commit, viewport-dependent cluster activation during navigation, inset-placement convergence, and per-inset rendering. Each scripted benchmark was repeated at least five times with approximately 1,000 seed states per dataset. All measurements were taken in Google Chrome 151 running a production build of the application on a laptop with an AMD Ryzen 9 5900HX (8 cores, 16 threads), 16GB RAM, and the processor's integrated AMD Radeon graphics driving a 2560×1440 display at 165 Hz. Interactions were scripted through the browser's automation protocol so that every dataset received an identical interaction sequence.

All blocking stages remained interactive. A selection or query commit computes the full recursive model, building the DoI distance field once and running the alternation of trajectory and spatial propagation to its fixed point, which took 67–89 ms at the median across our datasets (worst observed run 96.5 ms, at most 10 alternation rounds). A released slider re-runs only the alternation over the already computed field, which required under 1 ms on datasets without trajectories and 52–67 ms with them. Slider drags preview the same converged result per tick in GPU fragment passes while a worker streams the exact field in behind the preview; this sustained 110–162 fps on the benchmark machine's integrated GPU. On common 60 Hz displays, dragging is thus display-limited for every dataset. Committing a text query took 50–91 ms, and cluster activation 1.1–2.8 ms at the median.

Only placement convergence exceeded 100 ms. However, this matters little in practice, as insets reach a near-final layout almost immediately, and the remaining settling (median 1.6–3.7 s at the tabled inset counts) is minor positional refinement that runs non-blocking while the user continues to zoom and pan. Per-inset rendering stays below 5 ms even for aggregates of tens of thousands of states, making placement rather than content generation the limiting factor.

Taken together, these results mean the complete workflow keeps pace with the user, and exploring, querying, and dragging sliders remains smooth on every dataset we evaluated.

VIII. DISCUSSION

We now revisit the shift from select-and-inspect workflows to progressive disclosure, then delineate the scope of our method and outline directions toward continuous detail, larger datasets, and broader evaluation.

A. From Select-and-Inspect to Progressive Disclosure

Predefined labels and hierarchies support efficient navigation when they already encode the distinction an analyst seeks. Their limitation appears when the target is absent from the label vocabulary or lies at a granularity level the hierarchy does not provide. Workflows in prior work then revert to repeated selection and inspection or cannot reach the insight at all. The same principle applies to trajectory analysis, where manually defining and connecting groups can be avoided when the relevant sequence is disclosed directly through navigation.

Our central observation is that automatically deriving both in-place, high-dimensional annotations and their hierarchy from the data turns such workflows into progressive disclosure at the granularity required by the current view. Dynamically generated labels could serve the same role. Navigation can therefore reveal clusters, subclusters, individual states, and trajectory structure without repeatedly opening separate detail views. Insets serve simultaneously as explanations of the current structure and as navigational cues. These insights could generalize to focus+context visualization systems and multiple coordinated views that rely on repeated search-and-inspect workflows.

However, detail views remain dataset-specific and must be designed once for each data type to expose the relevant high-dimensional information.

B. Scope

Our method assumes a fixed 2D projection and contents that can be summarized or contrasted. Within this scope, the same workflow applies to image collections, video frames, reinforcement-learning states, game positions, and tabular data.

Although framed here for projection spaces, the workflow may also extend to scatterplots more broadly and to node-link representations of graphs. Adapting it to these structures is an interesting direction for future work.

C. Toward Continuous and Feature-Aware Detail

The current prototype discretizes both relevance and representation: states enter DoI buckets, and clusters receive either labels or insets. A more continuous mapping could gradually resolve a label into a compact statistic, summary inset, and increasingly detailed representation as relevance and available space increase.

Data DoI is also currently assigned to complete states. A per-feature DoI could preserve *why* a state became relevant, allowing summaries and differences to emphasize the pixels, attributes, actions, or transition properties that matched the analyst's intent. Richer specifications, including action sequences, spatial trajectory patterns such as oscillation or stagnation, and interaction-history-based suggestions, could express that intent more directly.

D. Scalability

The current scalability bottleneck is the client-only implementation rather than the multiscale visual abstraction. For datasets approaching 10^6 states, a client-server design could retain only columnar geometry, trajectory membership, and DoI values on the client, while a data service stores feature payloads and returns aggregated insets on demand. Viewport-dependent hierarchy levels and trajectory geometry could likewise be streamed by tile, making client memory depend primarily on visible content.

Our runtime evaluation covers interactive use of the datasets exemplified in this paper. Larger datasets may additionally require progressive loading, coarser propagation structures, and staged placement.

E. Evaluation Limits

Finally, the qualitative results inspection was performed by the authors, since which insights each method can surface and how much interaction they require are properties of the methods rather than of the operator. How the resulting annotations are interpreted, how the propagation parameters are steered, and how both integrate into analytical practice across users and domains remains to be assessed in user studies.

IX. CONCLUSION

We presented an adaptive workflow that enriches projections of high-dimensional data with in-place annotations. Interest specified through queries, selections, and navigation is propagated through the projection as data DoI, while layout DoI combines it with hierarchical cluster structure and view-dependent factors to select annotation targets. These are translated into text labels, single-item insets, summary insets, and difference insets placed automatically within the scatterplot. The resulting views refine and coarsen smoothly under zoom and pan, in the manner of an annotated map, for both trajectory data and unconnected items. Across usage scenarios and an evaluation, we showed that the workflow reproduces insights from prior projection-analysis work with

fewer workflow steps. More importantly, our evaluation illustrated that data-derived, in-place annotations from dynamically generated hierarchies turn repeated select-and-inspect loops into progressive disclosure.

Each stage of the workflow, from interest specification to annotation design, remains extensible. More generally, our findings suggest that detail views can serve not only to expose richer information after selection but also act as important navigational cues within an overview. Treating interest as an explicit, propagated quantity and annotations as its budgeted in-place response offers a recipe for showing details where they matter, and in the right context.

SUPPLEMENTAL MATERIALS

All supplemental materials are available at <https://jku-vds-lab.at/tvcg26-steinparz/material/>: (i) a demo video, (ii) the open-source prototype and pip-installable Jupyter widget, and (iii) links to reproduce the use cases and evaluations. Video and figures are released under CC BY 4.0; code under its repository license.

ACKNOWLEDGMENTS

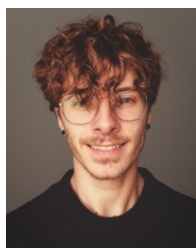
We thank the University of Edinburgh for the use of the video and ground truth data. This work was supported in part by the Austrian Science Fund (FWF) [DFH 23-N] and [10.55776/P36453].

Declaration of generative AI: The authors used OpenAI's ChatGPT (GPT-5) and Anthropic's Claude (Opus 4 and Fable 5) as writing assistants. The tools were used in an iterative human-in-the-loop manner to suggest rephrasings and improve grammar and structure, similar to a rubber-duck debugging process for prose, as well as for formatting and consistency checks. All AI-generated text was critically reviewed, edited, and rewritten by the authors. No generative AI tools were used to create or modify the data or analysis results. The figures show author-produced tool output, with AI assistance limited to figure layout scripting.

REFERENCES

- [1] L. v. d. Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.
- [2] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," *arXiv preprint arXiv:1802.03426*, 2018.
- [3] L. G. Nonato and M. Aupetit, "Multidimensional projection for visual analytics: Linking techniques with distortions, tasks, and layout enrichment," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 8, pp. 2650–2673, 2019.
- [4] G. W. Furnas, "The FISHEYE view: A new look at structured files," Bell Laboratories, New York, Technical memorandum, 1981.
- [5] C. A. Steinparz, Y. Metz, A. Hinterreiter, M. Waldner, M. El-Assady, and M. Streit, "Parametric detail-on-demand for projection spaces of high-dimensional data," in *Joint Proceedings of the ACM UMAP 2026 Workshops*, ser. CEUR Workshop Proceedings, vol. 4206, 2026, pp. 249–252. [Online]. Available: <https://ceur-ws.org/Vol-4206/ParaAdapt-2.pdf>
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [7] A. M. Horst, A. P. Hill, and K. B. Gorman, *palmerpenguins: Palmer Archipelago (Antarctica) penguin data*, 2020, r package version 0.1.0. [Online]. Available: <https://allisonhorst.github.io/palmerpenguins/>

- [8] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
- [9] M. D. Rahman, G. J. Quadri, B. Doppalapudi, D. A. Szafir, and P. Rosen, "A qualitative analysis of common practices in annotations: A taxonomy and design space," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 1, pp. 360–370, 2025.
- [10] A. Sarikaya, M. Gleicher, and D. A. Szafir, "Design factors for summary visualization in visual analytics," *Computer Graphics Forum*, vol. 37, no. 3, pp. 145–156, 2018.
- [11] K. Eckelt, A. Hinterreiter, P. Adelberger, C. Walchshofer, V. Dhanoa, C. Humer, M. Heckmann, C. A. Steinparz, and M. Streit, "Visual exploration of relationships and structure in low-dimensional embeddings," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 7, pp. 3312–3326, 2023.
- [12] J. Heer and B. Shneiderman, "Interactive dynamics for visual analysis," *Commun. ACM*, vol. 55, no. 4, pp. 45–54, Apr. 2012.
- [13] F. Lekschas, M. Behrisch, B. Bach, P. Kerpedjiev, N. Gehlenborg, and H. Pfister, "Pattern-driven navigation in 2D multiscale visualizations with Scalable Insets," *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 1, pp. 611–621, 2020.
- [14] R. Pless, "Image spaces and video trajectories: using Isomap to explore video sequences," in *Proceedings Ninth IEEE International Conference on Computer Vision*, 2003, pp. 1433–1440 vol.2.
- [15] B. Bach, C. Shi, N. Heulot, T. Madhyastha, T. Grabowski, and P. Dragicevic, "Time Curves: Folding time to visualize patterns of temporal evolution in data," *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 1, pp. 559–568, 2016.
- [16] A. Hinterreiter, C. A. Steinparz, M. Heckmann, H. Stitz, and M. Streit, "Projection Path Explorer: Exploring visual patterns in projected decision-making paths," *ACM Transactions on Interactive Intelligent Systems*, vol. 11, no. 3–4, p. Article 22, 2021.
- [17] T. Zahavy, N. Ben-Zrihem, and S. Mannor, "Graying the black box: Understanding DQNs," in *International conference on machine learning*. PMLR, 2016, pp. 1899–1908.
- [18] Y. Metz, E. Bykovets, L. Joos, D. Keim, and M. El-Assady, "VISITOR: Visual interactive state sequence exploration for reinforcement learning," *Computer Graphics Forum*, vol. 42, no. 3, pp. 397–408, 2023.
- [19] J. Stahnke, M. Dörk, B. Müller, and A. Thom, "Probing projections: Interaction techniques for interpreting arrangements and errors of dimensionality reductions," *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 1, pp. 629–638, 2016.
- [20] D. Ren, F. Hohman, H. Lin, and D. Moritz, "Embedding Atlas: Low-friction, interactive embedding visualization," in *2025 IEEE Visualization and Visual Analytics (VIS)*, 2025, pp. 191–195.
- [21] N. Grossmann, E. Gröller, and M. Waldner, "Concept Splatters: Exploration of latent spaces based on human interpretable concepts," *Computers & Graphics*, vol. 105, pp. 73–84, 2022.
- [22] D. Smilkov, N. Thorat, C. Nicholson, E. Reif, F. B. Viégas, and M. Wattenberg, "Embedding projector: Interactive visualization and interpretation of embeddings," 2016. [Online]. Available: <https://arxiv.org/abs/1611.05469>
- [23] F. van Ham and A. Perer, "search, show context, expand on demand: Supporting large graph exploration with degree-of-interest," *IEEE Transactions on Visualization and Computer Graphics*, vol. 15, no. 6, pp. 953–960, 2009.
- [24] N. Elmqvist and J.-D. Fekete, "Hierarchical aggregation for information visualization: Overview, techniques, and design guidelines," *IEEE Transactions on Visualization and Computer Graphics*, vol. 16, no. 3, pp. 439–454, 2009.
- [25] J. Abello, S. Hadlak, H. Schumann, and H.-J. Schulz, "A modular degree-of-interest specification for the visual analysis of large dynamic networks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 20, no. 3, pp. 337–350, 2014.
- [26] K. Perlin and D. Fox, "Pad: An alternative approach to the computer interface," in *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '93)*, 1993, pp. 57–64.
- [27] B. B. Bederson and J. D. Hollan, "Pad++: A zooming graphical interface for exploring alternate interface physics," in *Proceedings of the 7th Annual ACM Symposium on User Interface Software and Technology (UIST '94)*, 1994, pp. 17–26.
- [28] A. Cockburn, A. Karlson, and B. B. Bederson, "A review of overview+detail, zooming, and focus+context interfaces," *ACM Comput. Surv.*, vol. 41, no. 1, Jan. 2009.
- [29] C. Tominski, S. Gladisch, U. Kister, R. Dachselt, and H. Schumann, "Interactive lenses for visualization: An extended survey," *Computer Graphics Forum*, vol. 36, no. 6, pp. 173–200, 2017.
- [30] H. Doleisch and H. Hauser, "Smooth brushing for focus+context visualization of simulation data in 3D," *Journal of WSCG*, vol. 10, no. 1, pp. 147–154, 2002.
- [31] B. Bach, N. H. Riche, C. Hurter, K. Marriott, and T. Dwyer, "Towards unambiguous edge bundling: Investigating confluent drawings for network visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 23, no. 1, pp. 541–550, 2016.
- [32] D. Holten, "Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data," *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, no. 5, pp. 741–748, 2006.
- [33] K. Xu, C. Rooney, P. Passmore, D.-H. Ham, and P. H. Nguyen, "A user study on curved edges in graph visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 18, no. 12, pp. 2449–2456, 2012.
- [34] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*. AAAI Press, 1996, pp. 226–231.
- [35] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, ser. Lecture Notes in Computer Science, vol. 7819. Springer, 2013, pp. 160–172.
- [36] C. Collins, G. Penn, and S. Carpendale, "Bubble sets: Revealing set relations with isocontours over existing visualizations," *IEEE Transactions on Visualization and Computer Graphics*, vol. 15, no. 6, pp. 1009–1016, 2009.
- [37] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance transforms of sampled functions," *Theory of Computing*, vol. 8, no. 1, pp. 415–428, 2012.
- [38] M. Formann and F. Wagner, "A packing problem with applications to lettering of maps," in *Proceedings of the seventh annual symposium on Computational geometry*, 1991, pp. 281–288.
- [39] School of Informatics, University of Edinburgh, "Edinburgh office monitoring video dataset," 2020, <https://homepages.inf.ed.ac.uk/rbf/OFFICEDATA/>, accessed 2026-07-05.
- [40] T. Qasim, R. B. Fisher, and N. Bhatti, "Ground-truthing large human behavior monitoring datasets," in *2020 25th International Conference on Pattern Recognition (ICPR)*, 2021, pp. 2763–2770.
- [41] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," 2017.



Christian A. Steinparz is a PhD student in the Visual Data Science Lab led by Marc Streit at Johannes Kepler University (JKU) Linz, Austria. He received a BSc in Computer Science and an MSc in Artificial Intelligence from JKU Linz. During his PhD, he spent a research stay in the Interactive Visualization and Intelligence Augmentation group of Mennatalah El-Assady at ETH Zurich. His research interests include visualization, explainable AI, reinforcement learning, and dimensionality reduction.



Yannick Metz is a postdoctoral researcher at the IVIA Lab of Prof. Menna El-Assady at ETH Zürich. He did his PhD at the University of Konstanz under the main supervision of Prof. Daniel Keim. His work focuses on interactive machine learning, reinforcement learning, and training agents with expressive human feedback. He furthermore contributed to visual analytics research for domains such as medicine, agriculture and civil security.



Andreas Hinterreiter is a postdoctoral researcher and University Assistant in the Visual Data Science Lab at Johannes Kepler University (JKU) Linz. During his PhD, which he finished in 2022, he spent a year at the Biomedical Image Analysis Group at Imperial College London. His research focuses on dimensionality reduction and explainable AI.



Mennatallah El-Assady is an assistant professor with the Department of Computer Science, ETH Zürich. She heads the Interactive Visualization and Intelligence Augmentation (IVIA) Lab. Prior to that, she was a research fellow with the ETH AI Center; and before that, she was a research associate in the group for Data Analysis and Visualization with the University of Konstanz (Germany) and in the Visualization for Information Analysis Lab with the OntarioTech University (Canada). Her main research interest is studying interactive human-AI collaboration interfaces for effective problem-solving and decision-making.



Manuela Waldner is an associate professor at the Institute of Visual Computing & Human-Centered Technology at TU Wien, Austria. She has a PhD from Graz University of Technology. In 2014, she received a fellowship for highly qualified female scientists by the Austrian Science Fund (FWF). Her main research interest is scalable and interactive data analysis and visualization.



Marc Streit is a Full Professor of Visual Data Science at the Johannes Kepler University Linz, Austria. He finished his PhD at the Graz University of Technology in 2011. His scientific areas of interest include visualization, visual analytics, and explainable AI. He won multiple best paper and runner-up awards at InfoVis, EuroVis, and CHI. Marc is also co-founder of datavisyn, a spin-off company developing data visualization solutions for applications in pharmaceutical and biomedical R&D.